Track A – Ateko Project: Concurrency and Transactions

Introduction

In Assignment 3, you implemented a backend service for a simplified e-commerce platform using **FastAPI** and **SQLAlchemy**. The system allowed clients to browse products, create orders, retrieve analytics, and interact with the database through REST APIs.

As part of that assignment, you implemented a **multi-step order creation process**, which included operations such as:

- validating the customer
- validating products
- checking product stock
- deducting stock
- creating an order
- inserting order items
- calculating the total amount

For the purpose of Assignment 3, you were intentionally instructed **not to implement transaction management**. The goal was to focus on building the core API functionality. However, this simplified implementation exposes an important real-world problem: **concurrency issues**.

In production systems, many users may interact with the backend simultaneously. Multiple API requests can arrive at the same time and attempt to modify the same database records. Without proper transaction control, these concurrent operations may produce incorrect or inconsistent results. For example:

- Two customers may try to purchase the same product at the same time.
- Both requests may read the same stock value before either request updates it.
- This can result in **overselling inventory**, even when there was not enough stock available.

To prevent such problems, modern backend systems rely on **database transactions and isolation mechanisms**. Transactions ensure that multiple database operations behave as a **single logical unit of work**, meaning either all steps succeed together or none of them are applied.

This assignment builds directly on your previous implementation. You will analyze the behavior of your existing system under concurrent access and improve it using **transaction management and concurrency reasoning**.

How to Start

To complete this assignment, you must continue using the **same project you built in Assignment 3**.

Locate your previous implementation of the **Ateko Mini Commerce Backend** and ensure that your application runs correctly using:

```
uvicorn main:app --reload
```

You will modify your existing code and answer several short conceptual questions based on the behavior of your system. You do **not need to rebuild the project**. The goal of this assignment is to improve the **correctness and safety of your existing API implementation**.

Part 1 – Concurrency Scenario Analysis

Consider the following situation in your e-commerce system. Initial database state:

```
Product ID: 15
Stock: 5
```

Two customers place orders at the same time.

Request A	Request B
POST /orders product_id = 15 quantity = 5	POST /orders product_id = 15 quantity = 3

Both requests reach the server at nearly the same time. Because your Week 3 implementation did not include transaction management, both requests may read the same stock value before either request updates the database.

Questions

Create a file named `Part1_Answers.txt` in your week4_assignment git repository. Answer the following questions.

1. What incorrect result could occur if both requests execute concurrently?
2. What type of problem is this (race condition, dirty read, lost update, etc.)?
3. Why does this issue occur in the Week 3 implementation?

Your answer should be concise (approximately **4–6 sentences total**).

Part 2 – Implement Transaction Protection

You must now improve your **POST /orders** endpoint to ensure that the order creation process behaves as a **single atomic operation**. If any step of the order creation process fails, the database must **rollback all changes**.

For example:

- If stock validation fails
- If order item creation fails
- If any database operation raises an exception

then **no partial data should remain in the database**.

Requirements

Modify your order creation logic so that the following operations execute inside a **single database transaction**:

1. Validate customer existence
2. Validate product existence
3. Check stock availability
4. Deduct stock
5. Insert order record
6. Insert order items
7. Calculate total amount

You may implement transaction handling using SQLAlchemy. Example pattern:

```
with db.begin():
    # perform database operations
or
try:
    db.begin()

    # operations

    db.commit()
except:
    db.rollback()
```

Your final implementation should ensure that **either the entire order succeeds or nothing is written to the database**. Put a copy of your updated implementation with some comments in your week4_assignment git repository.

Part 3 – Isolation Level Reasoning

Transactions can operate under different **isolation levels**, which control how concurrent transactions interact with each other.

Create a file named `Part3_Answers.txt` in your week4_assignment git repository. Answer the following questions.

1. Why would **READ UNCOMMITTED** be dangerous in an e-commerce system?
2. What type of issue does **REPEATABLE READ** help prevent?
3. Why might **SERIALIZABLE** provide stronger safety but reduce system performance?

Each answer should be **2–3 sentences**.

Part 4 – Concurrency in Real Systems

Imagine that **100 users submit orders at the same time** through your API.

Answer the following questions in a file named `Part4_Answers.txt` in your week4_assignment repository.

1. How do transactions help protect the database during concurrent order creation?
2. What role does connection pooling play when many users access the API simultaneously?
3. What problems might occur if the system had **no transactions and no connection pooling**?

Provide short explanations (2–4 sentences each).

Submission Instructions

Your submission must include:

- Your updated backend implementation
- The following files in your git repository:

`Part1_Answers.txt`

```
Part2.py
Part3_Answers.txt
Part4_Answers.txt
```

Ensure that your application still runs successfully using the following command and then share your git repository link with me on Brightspace under assignment 4 or through emailing me.

```
uvicorn main:app --reload
```